

A4: Multi-Agent Coding

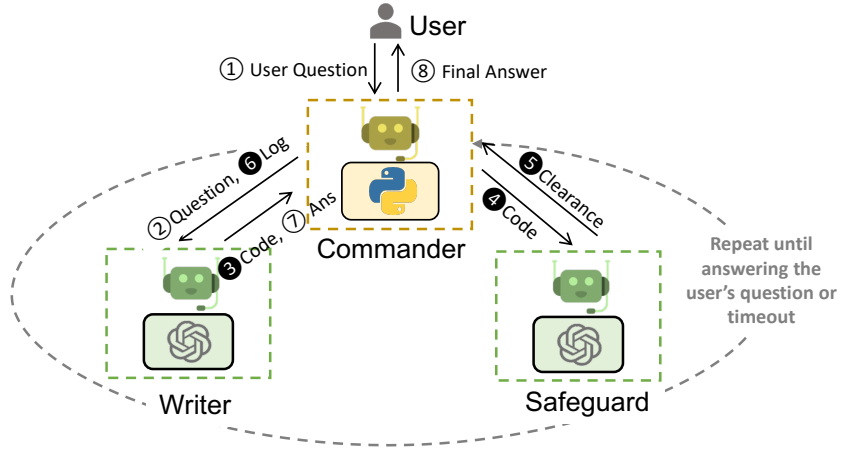


Figure 11: Our re-implementation of *OptiGuide* with AutoGen streamlining agents’ interactions. The Commander receives user questions (e.g., What if we prohibit shipping from supplier 1 to roastery 2?) and coordinates with the Writer and Safeguard. The Writer crafts the code and interpretation, the Safeguard ensures safety (e.g., not leaking information, no malicious code), and the Commander executes the code. If issues arise, the process can repeat until resolved. Shaded circles represent steps that may be repeated multiple times.

Detailed Workflow. The workflow can be described as follows. The end user initiates the interaction by posing a question, such as “What if we prohibit shipping from supplier 1 to roastery 2?”, marked by ① to the Commander agent. The Commander manages and coordinates with two LLM-based assistant agents: the Writer and the Safeguard. Apart from directing the flow of communication, the Commander has the responsibility of handling memory tied to user interactions. This capability enables the Commander to capture and retain valuable context regarding the user’s questions and their corresponding responses. Such memory is subsequently shared across the system, empowering the other agents with context from prior user interactions and ensuring more informed and relevant responses.

In this orchestrated process, the Writer, who combines the functions of a “Coder” and an “Interpreter” as defined in (Li et al., 2023a), will craft code and also interpret execution output logs. For instance, during code writing (② and ③), the Writer may craft code “`model.addConstr(x[‘supplier1’, ‘roastery2’] == 0, ‘prohibit’)`” to add an additional constraint to answer the user’s question.

After receiving the code, the Commander will communicate with the Safeguard to screen the code and ascertain its safety (④); once the code obtains the Safeguard’s clearance, marked by ⑤, the Commander will use external tools (e.g., Python) to execute the code and request the Writer to interpret the execution results for the user’s question (⑥ and ⑦). For instance, the writer may say “if we prohibit shipping from supplier 1 to roastery 2, the total cost would increase by 10.5%.” Bringing this intricate process full circle, the Commander furnishes the user with the concluding answer (⑧).

If at a point there is an exception - either a security red flag raised by Safeguard (in ⑤) or code execution failures within Commander, the Commander redirects the issue back to the Writer with essential information in logs (⑥). So, the process from ③ to ⑥ might be repeated multiple times, until each user query receives a thorough and satisfactory resolution or until the timeout. This entire complex workflow of multi-agent interaction is elegantly managed via AutoGen.

The core workflow code for *OptiGuide* was reduced from over 430 lines to 100 lines using AutoGen, leading to significant productivity improvement. The new agents are customizable, conversable, and can autonomously manage their chat memories. This consolidation allows the coder and interpreter roles to merge into a single “Writer” agent, resulting in a clean, concise, and intuitive implementation that is easier to maintain.

Manual Evaluation Comparing ChatGPT + Code Interpreter and AutoGen-based OptiGuide. ChatGPT + Code Interpreter is unable to execute code with private or customized dependencies (e.g., Gurobi), which means users need to have engineering expertise to manually handle multiple steps, disrupting the workflow and increasing the chance for mistakes. If users lack access or expertise, the burden falls on supporting engineers, increasing their on-call time.

We carried out a user study that juxtaposed OpenAI’s ChatGPT coupled with a Code Interpreter against AutoGen-based OptiGuide. The study focused on a coffee supply chain scenario, and an expert Python programmer with proficiency in Gurobi participated in the test. We evaluated both systems based on 10 randomly selected questions, measuring time and accuracy. While both systems answered 8 questions correctly, the Code Interpreter was significantly slower than OptiGuide because the former requires more manual intervention. On average, users needed to spend 4 minutes and 35 seconds to solve problems with the Code Interpreter, with a standard deviation of approximately 2.5 minutes. In contrast, OptiGuide’s average problem-solving time was around 1.5 minutes, most of which was spent waiting for responses from the GPT-4 model. This indicates a 3x saving on the user’s time with AutoGen-based OptiGuide.

While using ChatGPT + Code Interpreter, users had to read through the code and instructions to know where to paste the code snippets. Additionally, running the code involves downloading it and executing it in a terminal, a process that was both time-consuming and prone to errors. The response time from the Code Interpreter is also slower, as it generates lots of tokens to read the code, read the variables line-by-line, perform chains of thought analysis, and then produce the final answer code. In contrast, AutoGen integrates multiple agents to reduce user interactions by 3 - 5 times on average as reported in Table 4, where we evaluated our system with 2000 questions across five OptiGuide applications and measured how many prompts the user needs to type.

Table 4: Manual effort saved with OptiGuide (W/ GPT-4) while preserving the same coding performance is shown in the data below. The data include both the mean and standard deviations (indicated in parentheses).

Dataset	netflow	facility	tsp	coffee	diet
Saving Ratio	3.14x (0.65)	3.14x (0.64)	4.88x (1.71)	3.38x (0.86)	3.03x (0.31)

Table 13 and 15 provide a detailed comparison of user experience with ChatGPT+Code Interpreter and AutoGen-based OptiGuide. ChatGPT+Code Interpreter is unable to run code with private packages or customized dependencies (such as Gurobi); as a consequence, ChatGPT+Code Interpreter requires users to have engineering expertise and to manually handle multiple steps, disrupting the workflow and increasing the chance for mistakes. If customers lack access or expertise, the burden falls on supporting engineers, increasing their on-call time. In contrast, the automated chat by AutoGen is more streamlined and autonomous, integrating multiple agents to solve problems and address concerns. This results in a 5x reduction in interaction and fundamentally changes the overall usability of the system. A stable workflow can be potentially reused for other applications or to compose a larger one.

Takeaways: The implementation of the multi-agent design with AutoGen in the OptiGuide application offers several advantages. It simplifies the Python implementation and fosters a mixture of collaborative and adversarial problem-solving environments, with the Commander and Writer working together while the Safeguard acts as a virtual adversarial checker. This setup allows for proper memory management, as the Commander maintains memory related to user interactions, providing context-aware decision-making. Additionally, role-playing ensures that each agent’s memory remains isolated, preventing shortcuts and hallucinations

A5: Dynamic Group Chat

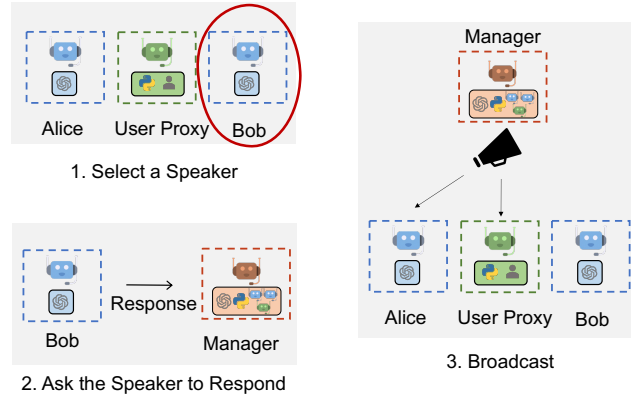


Figure 12: A5: Dynamic Group Chat: Overview of how AutoGen enables dynamic group chats to solve tasks. The Manager agent, which is an instance of the `GroupChatManager` class, performs the following three steps—select a single speaker (in this case Bob), ask the speaker to respond, and broadcast the selected speaker’s message to all other agents

To validate the necessity of multi-agent dynamic group chat and the effectiveness of the role-play speaker selection policy, we conducted a pilot study comparing a four-agent dynamic group chat system with two possible alternatives across 12 manually crafted complex tasks. An example task is “How much money would I earn if I bought 200 \$AAPL stocks at the lowest price in the last 30 days and sold them at the highest price? Save the results into a file.” The four-agent group chat system comprised the following group members: a user proxy to take human inputs, an engineer to write code and fix bugs, a critic to review code and provide feedback, and a code executor for executing code. One of the possible alternatives is a two-agent system involving an LLM-based assistant and a user proxy agent, and another alternative is a group chat system with the same group members but a task-based speaker selection policy. In the task-based speaker selection policy, we simply append role information, chat history, and the next speaker’s task into a single prompt. Through the pilot study, we observed that compared with a task-style prompt, utilizing a role-play prompt in dynamic speaker selection often leads to more effective consideration of both conversation context and role alignment during the process of generating the subsequent speaker, and consequently a higher success rate as reported in Table 5, fewer LLM calls and fewer termination failures, as reported in Table 6.

Table 5: Number of successes on the 12 tasks (higher the better).

Model	Two Agent	Group Chat	Group Chat with a task-based speaker selection policy
GPT-3.5-turbo	8	9	7
GPT-4	9	11	8

Table 6: Average # LLM calls and number of termination failures on the 12 tasks (lower the better).

Model	Two Agent	Group Chat	Group Chat with a task-based speaker selection policy
GPT-3.5-turbo	9.9, 9	5.3, 0	4, 0
GPT-4	6.8, 3	4.5, 0	4, 0

Task description

Write a script to download all the pdfs from arxiv in last three days and save them under /arxiv folder.

(a) two-agent chat

User:
Write a script to download all the pdfs from arxiv in last three days and save them under /arxiv folder.

Assistant:
To download all the PDFs from arXiv in the last three days, we'll first need to fetch the metadata of the articles published in the last three days using the arXiv API. Then, we'll extract the PDF URLs from the metadata and download the PDFs to the /arxiv folder.

Here's a Python script that does this. Save the script as `download_arxiv_pdfs.py` and execute it.

```
```python
(pytho n code)
```
```

After executing the script, all the PDFs from arXiv in the last three days will be downloaded and saved under the /arxiv folder.

```
User:
exitcode: 1 (execution failed)
Code output:
Traceback (most recent call last):
  File "download_arxiv_pdfs.py", line 3, in <module>
    import requests
ModuleNotFoundError: No module named 'requests'
```

Assistant:
I apologize for the oversight. It seems that the 'requests' library is not installed on your system. Please install the 'requests' library by running the following command:

```
```sh
pip install requests
```
```

After installing the 'requests' library, please run the 'download_arxiv_pdfs.py' script again.

```
User:
exitcode: 0 (execution succeeded)
Code output:
(pip install log)
```

Assistant:
Great! Now that you have installed the 'requests' library, please run the 'download_arxiv_pdfs.py' script again. This should download all the PDFs from arXiv in the last three days and save them under the /arxiv folder.

```
User:
exitcode: 0 (execution succeeded)
Code output:
(pip install log)
```

Assistant:
It seems you've executed the command to install the 'requests' library a gain. Please run the 'download_arxiv_pdfs.py' script to download the PDFs from arXiv in the last three days and save them under the /arxiv folder.

User:
(no output)

Assistant
My apologies for the confusion in my previous response. Please run the `download_arxiv_pdfs.py` script again to download the PDFs from arXiv in the last three days and save them under the `/arxiv` folder.

User
(no output)

Assistant
My apologies for the confusion in my previous response. Please run the `download_arxiv_pdfs.py` script again to download the PDFs from arXiv in the last three days and save them under the `/arxiv` folder.

User
(no output)

```
... repeat until max_auto_reply hit
```

(b) group chat

Admin: Welcome to the group chat! Work together to resolve my task. I'll reply [TERMINATE] when conversion ended.

Admin:
Critic, if code running fail, you ask engineer to rewrite code.

Admin:
engineer, you write python code step by step to resolve my task.

Admin:
executor, you run python code from enginnerand report bug

Admin:
Write a script to download all the pdfs from arxiv in last three days and save them under /arxiv folder.

```
Engineer:
Part 1:
```sh
pip install arxiv
pip install requests
... (rest of code)
```

```

Executor:
exitcode: 1 (execution failed)
Code output:
Collecting arxiv
// pip install log ...
Traceback (most recent call last):
 File "", line 15, in <module>
 results = arxiv.query(query=query, max_results=100)
 ^^^^^^^^^^^^^
AttributeError: module 'arxiv' has no attribute 'query'

```

**Critic:**  
It seems like there is an issue with the 'arxiv' module, as it does not have the 'query' attribute. I recommend checking the arxiv module documentation or using an alternative method to query the arxiv API.

```
Engineer:
I apologize for the confusion. It seems I made a mistake in using
the 'arxiv' module. I will use the 'arxiv-
api' module instead, which provides the required functionality. Pl
ease find the updated code below.
...sh
(updated code)
```

```

Executor:
... (pip log)
Traceback (most recent call last):
 File "", line 3, in <module>
 import arxiv_api
ModuleNotFoundError: No module named 'arxiv_api'

```

**Critic:**  
I apologize for the confusion. It seems that there is no package named 'arxiv-api' available. I made a mistake in my previous response. Let's revert to the 'arxiv' package and modify the code accordingly.  
... (modified code)

```
Executor:
exitcode: 0 (execution)
```

Admin:  
[Terminate]

Figure 13: Comparison of two-agent chat (a) and group chat (b) on a given task. The group chat resolves the task successfully with a smoother conversation, while the two-agent chat fails on the same task and ends with a repeated conversation.

## A6: Conversational Chess

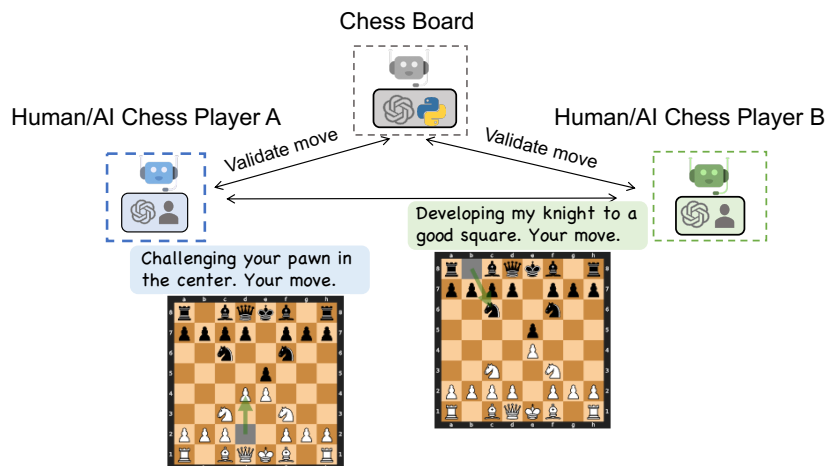


Figure 14: A6: Conversational Chess: Our conversational chess application can support various scenarios, as each player can be an LLM-empowered AI, a human, or a hybrid of the two. Here, the board agent maintains the rules of the game and supports the players with information about the board. Players and the board agent all use natural language for communication.

In Conversational Chess, each player is a AutoGen agent and can be powered either by a human or an AI. A third party, known as the board agent, is designed to provide players with information about the board and ensure that players' moves adhere to legal chess moves. Figure 14 illustrates the scenarios supported by Conversational Chess: AI/human vs. AI/human, and demonstrates how players and the board agent interact. This setup fosters social interaction and allows players to express their moves creatively, employing jokes, meme references, and character-playing, thereby making chess games more entertaining for both players and observers (Figure 15 provides an example of conversational chess).

To realize these scenarios, we constructed a player agent with LLM and human as back-end options. When human input is enabled, before sending the input to the board agent, it first prompts the human player to input the message that contains the move along with anything else the player wants to say (such as a witty comment). If human input is skipped or disabled, LLM is used to generate the message. The board agent is implemented with a custom reply function, which employs an LLM to parse the natural language input into a legal move in a structured format (e.g., UCI), and then pushes the move to the board. If the move is not legitimate, the board agent will reply with an error. Subsequently, the player agent needs to resend a message to the board agent until a legal move is made. Once the move is successfully pushed, the player agent sends the message to the opponent. As shown in Figure 15, the conversation between AI players can be natural and entertaining. When the player agent uses LLM to generate a message, it utilizes the board state and the error message from the board agent. This helps reduce the chance of hallucinating an invalid move. The chat between one player agent and the board agent is invisible to the other player agent, which helps keep the messages used in chat completion well-managed.

There are two notable benefits of using AutoGen to implement Conversational Chess. Firstly, the agent design in AutoGen facilitates the natural creation of objects and their interactions needed in our chess game. This makes development easy and intuitive. For example, the isolation of chat messages simplifies the process of making a proper LLM chat completion inference call. Secondly, AutoGen greatly simplifies the implementation of agent behaviors using composition. Specifically, we utilized the `register_reply` method supported by AutoGen agents to instantiate player agents and a board agent with custom reply functions. Concentrating the extension work needed at a single point (the reply function) simplifies the reasoning processes, and development and maintenance effort.